Product Requirements and Specification Document

Project Name

MedBotX - Advanced Medical Chatbot with Memory

Description

MedBotX is a healthcare chatbot leveraging LangChain and OpenAI, designed to answer patient queries, maintain conversational context, and support both temporary and permanent memory across sessions. The system is built with Python, FastAPI, and integrates generative AI for advanced, context-aware interactions.

---

1. Goals & Objectives

Goal	Description
Accurate Medical Q&A	Provide reliable, up-to-date answers to patient health queries
Contextual Memory	Maintain conversation context within and across sessions
User Data Security	Ensure privacy and security of patient data
Scalability	Support multiple concurrent users
Futuristic UX	Deliver a modern, research-oriented user experience

---

2. Core Features

Feature	Description
Conversational AI	Natural language understanding and response generation via OpenAI
Temporary Memory	Retain context during a single session
Permanent Memory	Store and retrieve user-specific context across sessions
Session Management	Unique session IDs, user authentication (optional)
API Interface	RESTful endpoints via FastAPI
Logging & Monitoring	Track interactions and system performance

---

3. Functional Requirements

ID	Requirement
FR1	The system shall accept and process user queries via REST API
FR2	The chatbot shall generate context-aware responses using OpenAI models

FR3	The system shall maintain temporary memory for each session
FR4	The system shall persist permanent memory for registered users
FR5	The system shall allow retrieval and update of user memory
FR6	The system shall log all interactions for monitoring and debugging
FR7	The system shall provide basic authentication for permanent memory features
FR8	The system shall ensure all data is encrypted in transit and at rest

---

4. Non-Functional Requirements

ID	Requirement
NFR1	Response time < 2 seconds for 95% of requests
NFR2	System uptime ≥ 99%
NFR3	Compliance with HIPAA or equivalent data privacy standards
NFR4	Modular, extensible codebase
NFR5	Support for at least 100 concurrent sessions

---

5. Technical Specifications

Component	Technology / Approach
Backend	Python 3.10+, FastAPI
AI Integration	LangChain, OpenAI GPT-4 API
Memory Storage	Temporary: In-memory (Redis or Python dict); Permanent: Database (PostgreSQL)
Authentication	JWT-based (optional for permanent memory)
Deployment	Docker, cloud-ready (AWS/GCP/Azure)
Monitoring	Logging (structured logs), Prometheus/Grafana (optional)

---

6. API Endpoints (Sample)

```
POST /chat
Request: { "session_id": "abc123", "message": "What are the symptoms of flu?" }
Response: { "response": "Common symptoms include fever, cough, and fatigue." }

POST /memory/save
Request: { "user_id": "u123", "data": { ... } }
Response: { "status": "success" }

GET /memory/load
```

```
Request: { "user_id": "u123" }
Response: { "data": { ... } }
```

---

7. User Flows

1. **Anonymous Session**
   - User starts chat → System assigns session ID → Maintains temporary memory → Session ends, memory cleared
2. **Registered User**
   - User authenticates → Starts chat → System loads permanent memory → Maintains context → Updates memory on session end

---

8. Security & Privacy

- All data encrypted in transit (HTTPS) and at rest
- No storage of sensitive data unless user is authenticated
- Compliance with healthcare data privacy standards

---

9. Success Metrics

Metric	Target Value
Response accuracy	≥ 90% (manual review)
Average response time	< 2 seconds
User retention (registered)	≥ 60% after 1 week
Uptime	≥ 99%

---

10. Out of Scope

- Diagnosis or treatment recommendations
- Integration with EHR/EMR systems
- Voice interface

---

11. Milestones

Milestone	Target Date
MVP API & Memory	Week 2
OpenAI Integration	Week 3
Permanent Memory	Week 4

Security & Compliance	Week 5
Testing & Deployment	Week 6

---

12. Appendix

References:

- LangChain Documentation
- OpenAI API Docs
- FastAPI Docs

End of Document